

ShadowClone: Scalable Decentralized Identity with Cross-Domain Anonymity and Accountable Traceability

Yizhong Liu*
liyizhong@buaa.edu.cn
Beihang University
Beijing, China

Na Wang†
nawang@buaa.edu.cn
Beihang University
Beijing, China

Zedan Zhao
zhaozedan@buaa.edu.cn
Beihang University
Beijing, China

Haojun Tan
tanhaojun25@buaa.edu.cn
Beihang University
Beijing, China

Boyu Zhao
boyuzhao@buaa.edu.cn
Beihang University
Beijing, China

Jianwei Liu†
liujianwei@buaa.edu.cn
Beihang University
Beijing, China

Abstract

Decentralized identity (DID) is a key infrastructure for Web3, granting users sovereign control over their private identity data. While existing DID systems like FADID-TT (WWW'25) realize anonymity and traceability within a single domain, the Web3 ecosystem is a multiverse of independent domains like DeFi, GameFi, and DAO. This multi-domain reality presents critical issues for current DID solutions. First, most existing solutions are built on the monolithic committee architecture, facing severe scalability bottlenecks as the committee size grows. Second, most existing solutions cannot offer strong cross-domain anonymity, where frequent cross-domain interaction inevitably exposes the user's privacy. Third, existing methods for tracing the identities of malicious users are inefficient.

To address these issues, we propose ShadowClone, a DID system that offers strong *scalability*, *cross-domain anonymity*, and *efficient accountable traceability*. Firstly, we introduce a sharding architecture that partitions the global system into multiple independent domain committees, enhancing system scalability. Moreover, we realize strong cross-domain anonymity, a new and stronger privacy guarantee for multi-domain DID systems. We design a domain-hiding presentation protocol based on an accountable ring signature scheme. A user can prove membership in a trusted set of domains without revealing home domain, preventing profiling based on domain affiliation. Furthermore, we design a hierarchical cross-domain tracing mechanism to ensure efficient accountable traceability. An opening committee first identifies the user's home domain and the home domain committee efficiently traces user identity within a smaller scope. Notably, the entire tracing process is distributed and publicly verifiable. Lastly, we conduct extensive experiments. The evaluation results demonstrate that ShadowClone realizes only the 10.4% and 20% registration and tracing latency

compared with FADID-TT, respectively, showcasing high scalability and tracing efficiency compared with FADID-TT while providing strong cross-domain anonymity and decentralization for Web3.

CCS Concepts

• **Security and privacy** → **Privacy-preserving protocols; Distributed systems security**; Cryptography.

Keywords

decentralized identity, cross-domain anonymity, accountable traceability, scalability

ACM Reference Format:

Yizhong Liu, Zedan Zhao, Boyu Zhao, Na Wang, Haojun Tan, and Jianwei Liu. 2026. ShadowClone: Scalable Decentralized Identity with Cross-Domain Anonymity and Accountable Traceability. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792424>

1 Introduction

Decentralized identity (DID) is emerging as a key infrastructure of the Web3 ecosystem [9, 26, 35], empowering users with sovereign control over their digital identities and credentials [3]. In DID systems, a user uses a unique self-generated identifier and attribute credentials issued by distributed authorities to prove that it has the qualification to access some services provided by the verifier [12]. Due to the great decentralized paradigm, DID systems are utilized in a broad range of functions in Web3, including the trustworthy digital asset ownership authentication and exchange [16, 21], the trust identity management [8, 41], the self-managed personal data showcasing in the social medias [32], etc. To support such functions, DID systems are required to achieve strong privacy protection within distinct and autonomous domains, such as DeFi platforms [1], GameFi applications [34], and social networks [11].

A fundamental privacy protection requirement in DID systems is *identity anonymity*, where all other entities (authorities and verifiers) in the DID systems cannot learn the legal user's real identity information. Furthermore, *unlinkability* is required across multiple service accesses, preventing legal users from being profiled based on their behavior patterns. Although most existing works [22, 28, 39] try to utilize distinct identifiers to achieve the identity anonymity or utilize decentralized anonymous credentials (DAC) to realize unlinkability, they cannot realize the two properties at the same time

*Also with Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, Beijing, the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Zhejiang, China.

†Corresponding author: Na Wang, Jianwei Liu



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792424>

Table 1: Comparison between ShadowClone and other related work

System	Scalability ¹		Strong Cross-Domain Anonymity			Efficient Accountable Traceability		Application to Decentralized Multi-Domain Web3
	User's request processing complexity	Committees storage overhead ²	Identity	Anonymity	Domain-hiding	Unlinkability	Threshold verifiable tracing	Tracing complexity
CanDID[28]	$O(m)$	$O(lmn)$ pk files	✓	✗	✗	✗	✓	$O(m)$
Coconut[39]	$O(m)$	$O(mn)$ pk files	✓ ³	✗	✓	✗	—	✗
TMAC[22]	$O(m)$	$O(mn)$ pk files	✗	✗	✓	✓ ⁴	$O(m)$	✗
FADID-TT[27]	$O(m)$	$O(mn)$ shares	✓	✗	✓	✓	$O(m)$	✗
ShadowClone (Ours)	$O(m/k)$	$O(mn/k)$ shares	✓	✓	✓	✓	$O(m/k)$	✓

¹ In this table, k is the number of domains, m is the total number of users in the system, n is the total number of the committee members in the system, l is the number of server providers.

² In CanDID, Coconut and TMAC, committee stores public key files with hundreds of bytes. In FADID-TT and ShadowClone, committee stores secret shares of user's identity with 32/64 bytes.

³ Coconut satisfies identity anonymity when not considering tracing. ⁴ TMAC achieves centralized tracing.

within a single domain. In addition, the DID systems should support *traceability*, allowing authorities to reveal the real identities of users who misbehave. As most existing solutions realize traceability via a centralized authority to search the entire list of user public keys or identities, the abuse of traceability still happens, undermining the accountability. Notably, FADID-TT [27] tries to provide identity anonymity and unlinkability by introducing the distributed identity committee and hiding the user real identity via secret sharing and anonymous signature. Besides, it can offer a threshold verifiable opening of malicious users to realize accountable traceability.

However, with the domain expansion of the Web3 ecosystem [15, 24], frequent cross-domain interactions introduce more security and performance issues in *decentralized multi-domain Web3*. Under such scenarios, the *scalability* of DID systems is crucial to ensure the efficiency of user request processing with lower storage overhead in multiple domains. As most DID solutions process user request serially and store user public key message globally, they cannot hold possible millions of users across domains. Although FADID-TT [27] utilizes secret sharing and anonymous signature to eliminate the management overhead of user public keys, it still adopts a monolithic committee architecture to store the users' identity secret shares and generate the users' private key from these shares, incurring a high processing complexity of $O(m)$ and committee storage of $O(mn)$ (m refers to the total number of users and n refers to the number of the committee members). Besides, in cross-domain interactions, revealing a user's affiliated domain may inadvertently disclose sensitive contextual information, such as organizational membership or service provider relationships. Such exposure not only compromises user anonymity but also enables adversaries to perform correlation attacks across multiple domains. Since most existing DID systems only ensure identity anonymity and unlinkability in a single domain, there is still a gap to ensure *strong cross-domain anonymity*. To achieve this property, DID solutions should further realize *domain-hiding*, where all other parties cannot learn the affiliated domain of legal users in cross-domain interactions. Furthermore, achieving *efficient accountable traceability* is essential to maintain trust and regulatory compliance without undermining user privacy. As most existing DID solutions need to realize traceability by sequential user search, the tracing complexity is extremely high, making it impractical for Web3 applications. Therefore, to satisfy growing privacy demands of massive users in decentralized multi-domain Web3 ecosystems, it is crucial to design a DID system that not only has high scalability, but also ensures strong cross-domain anonymity while maintaining efficient accountable traceability.

Our contributions. To address the above issues, we present ShadowClone, a DID system for the multi-domain Web3 multiverse that achieves scalability, strong cross-domain anonymity, and efficient

accountable traceability. We summarize our design and existing DID systems in Table 1, with detailed related work in Appendix C. Our contributions are as follows.

A sharding DID architecture for scalability. We propose a sharding architecture that partitions the global system into multiple autonomous distributed domain committees, each responsible only for the users within its domain. This distributed design is inherently aligned with the decentralized nature of Web3. And this sharding architecture directly addresses the scalability bottleneck of monolithic systems by distributing the computational and storage load. Compared with the state-of-the-art scheme FADID-TT (WWW'25), the processing complexity decreases from $O(m)$ to $O(m/k)$, and the committee's storage overhead reduces from $O(mn)$ to $O(mn/k)$.

Strong cross-domain anonymity through domain-hiding presentation. We design a domain-hiding presentation protocol to achieve strong cross-domain anonymity, a new and stronger privacy guarantee for multi-domain DID systems. This is enabled by decentralized anonymous credentials (DACs) and a novel accountable ring signature scheme. After distributed registration in the home domain, the user obtains the issued registration secret key bound to her real identity and the home domain. When accessing services, the user generates a one-time show token through DACs. Then, the user forms a ring of domain public keys (including her own) and cryptographically proves that she has registered validly in one of the domains, without revealing the real identity and home domain. This presentation protocol completely resists link analysis and prevents domain-level profiling based on domain affiliation.

Hierarchical decentralized tracing mechanism with efficiency and accountability. We design a hierarchical cross-domain tracing mechanism that is efficient, accountable, and fully decentralized. This tracing process is performed in two distributed stages. First, a global opening committee efficiently identifies only the malicious user's home domain by performing threshold decryption on an embedded ciphertext, without scanning any user list. Second, the tracing request is routed to the identified domain committee, which performs an efficient search on its much smaller local registry. This reduces the tracing complexity from $O(m)$ in FADID-TT to $O(m/k)$. The entire process is publicly verifiable through zero-knowledge proofs, ensuring accountability without any centralized authority.

Implementation and extensive experiments. We implement a prototype of ShadowClone and conduct a comprehensive evaluation, demonstrating a substantial performance improvement over the baseline DID systems FADID-TT [27]. The experiment results show that ShadowClone can realize only the 10.4% and 20% registration and tracing latency compared with FADID-TT in a 500-user system, achieving scalability and tracing efficiency. Furthermore, deployment on practical blockchains shows a reduction in on-chain costs, tracing operations require over 62% less time on Hyperledger

Fabric and 22% less gas on Ethereum, confirming its practical advantages for the modern Web3 ecosystem.

2 Core Building Blocks

For brevity, we present concise definitions of the core components essential for understanding our main contributions here. We defer the detailed formalisms and additional preliminaries (*bilinear pairings* and *Decentralized Anonymous Credentials*) to Appendix A.

2.1 Secret Sharing

Secret sharing schemes are fundamental to the distributed nature of ShadowClone. In a (n, t) threshold scheme, a secret s is split into n shares such that any t shares suffice for reconstruction, while fewer than t shares reveal nothing. We use Shamir's Secret Sharing (SSS) [37], comprising SSS.Share for splitting a secret into shares and SSS.Reconstruct for recovering the secret via Lagrange interpolation. See Appendix A.2 for formal definitions.

2.2 Non-Interactive Zero-Knowledge Proof

A Non-Interactive Zero-Knowledge (NIZK) proof allows a prover to convince a verifier that a statement is true without revealing secret information. We abstract its core functionalities into NIZK.Prove and NIZK.Verify, instantiated with Schnorr-style proofs [7]. See Appendix A.3 for more details.

2.3 Threshold Public Key Encryption

Threshold Public Key Encryption (TPKE) [5, 10, 38] distributes a secret key among multiple parties, requiring at least threshold parties to cooperate for decryption. It comprises four algorithms: TPKE.KeyGen for distributed key generation, TPKE.Enc for public key encryption, TPKE.DecShare for partial decryption, and TPKE.Combine for combining shares. See Appendix A.4 for details.

3 Problem Formulation

This section introduces the overall problem formulation and the high-level overview of our system. We first specify the design goals. Then, we describe the system and threat model, including the key roles in a multi-domain Web3 environment and the capabilities of potential adversaries. Finally, we present the system architecture and its workflow, illustrating how the different roles interact to achieve secure and accountable cross-domain identity management.

3.1 Design Goal

To meet the requirements of cross-domain decentralized identity in Web3, our design is guided by the following goals:

- **Decentralization.** To align with the foundational principles of Web3 ecosystem, the system must be inherently decentralized, preventing single point of failure or trust. This enhances fault tolerance and robustness against attacks. To achieve this, critical functions, including user registration process and the tracing mechanism, need operate in a distributed manner.
- **Scalability.** The system must be scalable to accommodate global-scale users. Its architecture should support a constantly growing number of users and domains while maintaining performance.
- **Strong Cross-Domain Anonymity.** A single token presentation by an honest user should be computationally indistinguishable from any other valid presentation, hiding the user's home domain and identity even when accessing services outside of it.

- **Accountable Traceability.** The system should enable a hierarchical and publicly verifiable tracing mechanism to address malicious behavior. First, identify the malicious user's home domain. Second, allow intra-domain trace by the responsible domain committee to reveal the user's real-world identity. Both stages should be publicly verifiable.
- **Unforgeability.** The system must prevent identity forgery and ensure that no party can impersonate a user or forge a credential presentation that can pass verification.

3.2 System and Threat Model

Roles. We consider a Web3 ecosystem composed of multiple autonomous domains, such as DeFi platforms, GameFi applications, or social networks, each managing its own users. The system involves the following roles:

- **Users.** A user is an entity associated with a unique real-world identifier (e.g., a national ID or Social Security Number). Each user registers in a single home domain but can access services across multiple domains. We assume two types of users: honest users, who seek privacy-preserving cross-domain authentication, and malicious users, who may exploit anonymity to commit fraud or disrupt services. The user set is dynamic and grows over time.
- **Domain Committees.** A domain committee is a decentralized committee responsible for managing all users within its domain. Each committee operates under a threshold security assumption. Its core function includes enabling anonymous and distributed user registration, collectively maintaining users' identity information, and enforcing intra-domain tracing for malicious user.
- **Verifiers.** A verifier is a service provider that verifies user identity and credentials before granting access. Examples include DeFi lending platforms and decentralized content providers. Verifiers correctly execute the verification protocol, but may try to compromise user privacy by linking user activities or inferring user home domain or real identity.
- **Opening Committee.** The opening committee is a distributed authority that supports cross-domain accountability by revealing the home domain of a malicious user. It can be composed of designated representatives from each domain committee while satisfying basic threshold assumption. It performs the first-stage tracing to identify a malicious user's home domain. Then, the responsible domain committee conducts a second-stage tracing to reveal the user's real world identity.
- **Credential Issuers.** Credential issuers are the decentralized authorities that issue anonymous credentials to registered users.

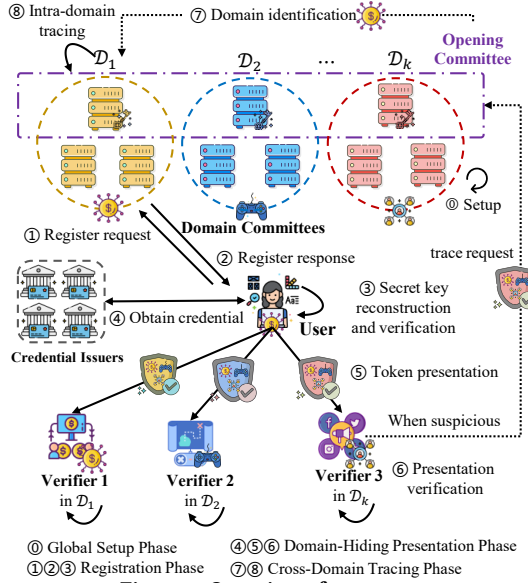
Threat Models. We assume a probabilistic polynomial-time adversary $\mathcal{A}$ with the following capabilities: (i) corrupt up to $t - 1$ nodes in any domain committee and up to $t_o - 1$ representatives in the opening committee; (ii) eavesdrop on all public communication channels; and (iii) obtain valid signatures from honest users.

The adversary's objectives are to (i) violate user anonymity by linking presentations, inferring a home domain, or de-anonymizing an identity; (ii) forge a valid presentation to impersonate legitimate users; (iii) destroy accountability by creating a presentation that cannot be traced or that is traced to incorrect home domains/users.

3.3 System Architecture and Workflow

System Architecture. The system consists of k independent domain committees $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_k$ and one opening committee $\mathcal{O}$.

Each domain committee collectively governs its users and maintains their associated cryptographic materials. Users obtain anonymous credentials from credential issuers and later generate cryptographic presentations to access services provided by verifiers. The opening committee coordinates with domain committees to enable accountable tracing when necessary.



Work Flow. Figure 1 illustrates the high-level workflow of our system, where the credentials component we instantiate with existing DAC systems. The workflow unfolds in four main phases: setup, registration, presentation, and tracing. The detailed procedures are given in Algorithm 1, 2, 3, 4 respectively.

- **Global Setup.** Initially, the system establishes a global cryptographic foundation for the entire multi-domain ecosystem (Figure 1, step ⑩). Global public parameters are generated and published. Each domain committee $\mathcal{D}_j$ distributively generates its own implicit secret key and the corresponding public committee verification key $cvk^{(\mathcal{D}_j)}$, which will be used for user registration within its domain. Concurrently, the opening committee also distributively generates its threshold opening secret key shares and a public opening key opk , which are used for the first stage of the tracing process.
- **Registration.** In this phase, a user establishes her identity within a single “home domain” for future authentication. Let’s consider a new user, Alice, who wants to join a DeFi platform. First, Alice prepares an identity registration request that is cryptographically bound to her unique real-world identifier s , and submits it to the DeFi domain committee (Figure 1, step ①). Then, the domain committee members collaboratively verify the request’s legitimacy and check their local records to ensure the identity s has not been previously registered. Upon successful validation, they issue shares of Alice’s registration secret key $ursk$ in a distributed manner (Figure 1, step ②). This secret key is uniquely bound to both Alice’s identity s and the domain committee’s secret key. Finally, Alice collects a sufficient number of these secret key shares. She locally reconstructs the complete registration secret key and verifies its correctness (Figure 1, step ③). Since the committee

operates under a threshold security assumption, Alice is guaranteed to reconstruct a valid key. After this process, Alice possesses a secret key for her identity within the DeFi domain. Notably, there is no corresponding user public key, enhancing privacy and reducing the burden of user’s public key management in system.

- **Domain-Hiding Presentation.** Once registered, a user can anonymously present credentials to access services across any domain without revealing their home domain or linking their activities. For example, as a user registered in DeFi platform, Alice may want to authenticate to a GameFi playing service without revealing her original domain membership. To do this, first, Alice obtains an anonymous credential from the credential issuers (Figure 1, step ④). Next, when Alice wants to access the GameFi service, she generates a one-time randomized show token from the credential. And she forms a ring (an anonymity set for the presentation) by selecting the public verification keys of several domains, including her own home domain and others. She then generates an accountable ring signature on the show token, which proves her membership in the ring of domains. This also embeds two pieces of information for potential tracing: a ciphertext c containing her home domain, and an intra-domain tracing tag T_s . The complete presentation is sent to the verifier (Figure 1, step ⑤). Finally, the verifier checks the presentation’s validity locally, without interacting with the credential issuers or any domain committee. If the show token and the accountable ring signature are valid, the user is authenticated and granted access (Figure 1, step ⑥). The verifier learns only that the user belongs to one of the domains in the ring and is legal, but not which one, thus preserving strong cross-domain anonymity.
- **Cross-Domain Tracing.** If a user misuses a service (e.g., engages in market manipulation in the DeFi platform), the system initiates a two-stage, publicly verifiable tracing procedure. When a presentation is flagged as suspicious, an authorized entity can submit a tracing request containing the malicious presentation to the opening committee. Upon validation, the two-stage tracing begins (Figure 1, step ⑦). First, in stage 1 (home domain identification), the opening committee members collaboratively perform threshold decryption on the ciphertext c from the suspicious presentation. This reveals the user’s home domain without exposing user’s specific identity. A cryptographic proof is attached to ensure this domain-opening process is publicly verifiable. Next, the intra-domain tracing request, along with the proof, is forwarded to the responsible domain committee. In stage 2 (intra-domain tracing), this responsible committee uses the tracing tag T_s to efficiently search its local user registry (Figure 1, step ⑧). By matching the tag, they can pinpoint the specific user. The committee can then collaboratively reconstruct her real-world identity. This final tracing step is also accompanied by a proof of correctness. The revealed identity and tracing evidence can be made public in system, effectively deterring future misconduct.

4 ShadowClone Design

Here, we present our detailed design of ShadowClone including four primary phases: *Global Setup* (Section 4.1, Algorithm 1), *Registration* (Section 4.2, Algorithm 2), *Domain-Hiding Presentation* (Section 4.3, Algorithm 3), and *Cross-Domain Tracing* (Section 4.4, Algorithm 4). Each phase addresses one aspect of the decentralized

identity life cycle in a multi-domain Web3 ecosystem: establishing cryptographic foundations, registering users, supporting anonymous service access, and ensuring efficient and accountable tracing. Notably, for the underlying decentralized anonymous credential (DAC) layer, our system is designed to be modular. We therefore leverage established DAC schemes [14, 17, 19, 36, 39]. This allows our system to focus on domain-hiding authentication and scalable tracing while remaining compatible with most of anonymous credential frameworks. We provide the comprehensive security analysis in Appendix B.

4.1 Global Setup

The Global Setup phase establishes the global cryptographic foundation for the multi-domain system, enabling secure interactions between system roles. The details are outlined in Algorithm 1.

Algorithm 1 Global Setup

Init: the system consists of k domains and one opening committee

Step ① Setup

- 1: **procedure** GLOBALSETUP(1^λ)
- 2: **generate** bilinear pairing params $bp = (p, \mathbb{G}, \tilde{\mathbb{G}}, \mathbb{G}_T, e, g, \tilde{g})$
- 3: **set** threshold t for each domain committee
- 4: **set** opening threshold t_o for the opening committee
- 5: **publish** $pp := (bp, t_d, t_o)$ $\triangleright$ global public parameters
- 6: $\triangleright$ opening committee distributedly generates opening keys
- 7: **run** $(opk, \{osk_i\}) \leftarrow \text{TPKE.KeyGen}(pp)$
- 8: $\triangleright$ each domain committee distributedly generates keys
- 9: **for each** domain $\mathcal{D}_j$ **do** $\triangleright$ each member in $\mathcal{D}_j$ obtains share $csk_i^{(\mathcal{D}_j)}$
- 10: **run** $(cvk^{(\mathcal{D}_j)}, \{csk_i^{(\mathcal{D}_j})\}) \leftarrow \text{DKG}(pp)$ $\triangleright cvk^{(j)} = g^{csk^{(\mathcal{D}_j)}}$
- 11: **publish** opk and all $cvk^{(\mathcal{D}_j)}$ globally

Step ①: Setup. This process takes security parameter 1^λ as input. First, it generates the global bilinear pairing parameters $bp = (p, \mathbb{G}, \tilde{\mathbb{G}}, \mathbb{G}_T, e, g, \tilde{g})$ (line 2). Here, $\mathbb{G}, \tilde{\mathbb{G}}, \mathbb{G}_T$ are finite groups with large prime order p . The mapping $e : \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \mathbb{G}_T$ represents a type-3 bilinear mapping, with g and $\tilde{g}$ as the generators of group $\mathbb{G}$ and $\tilde{\mathbb{G}}$ respectively. These parameters form the mathematical basis for all subsequent cryptographic operations in system. Next, it defines two distinct threshold values: t for each domain committee (line 3) and t_o for the opening committee (line 4). These thresholds specify the minimum number of committee members required to perform distributed user registration and threshold encryption, there by enhancing the fault tolerance.

Following the establishment of global public parameters $pp := (bp, t, t_o)$ (line 5), the system's distributed authorities generate their keys in a distributed manner. The opening committee executes a threshold public key encryption (TPKE) key generation protocol, TPKE.KeyGen, to produce a single opening public key opk and a set of corresponding secret key shares osk_i , with one share held by each member (line 6). This opening key pair is exclusively used for the first stage of tracing: identifying a malicious user's home domain. Concurrently, each of the k domain committee $\mathcal{D}_j$ independently performs a distributed key generation (DKG) protocol (line 7-8). This results in a unique public committee verification key $cvk^{(\mathcal{D}_j)}$ for each domain $\mathcal{D}_j$, along with a set of secret key shares $\{csk_i^{(\mathcal{D}_j})\}$ among its members. Here it satisfies $cvk^{(j)} = g^{csk^{(\mathcal{D}_j)}}$. These domain-specific key pairs are used for registering users and enabling intra-domain tracing. Finally, the opening public key opk and all domain verification keys $cvk^{(\mathcal{D}_j)}$ are published globally, making them accessible to all participants in the system (line 9).

4.2 Registration

The registration phase allows a user to establish a decentralized identity within a single chosen "home domain". This process involves distributed interactions between the user and the members of their home domain committee $\mathcal{D}_u$, as detailed in Algorithm 2.

Algorithm 2 Registration

Init: each domain $\mathcal{D}_j$ publishes its verification key $cvk^{(\mathcal{D}_j)}$.

Init: a user with her unique identity s in domain $\mathcal{D}_u$ wanted to be registered.

Let each member C_i in $\mathcal{D}_u$ store its domain's registered users in local table $\mathcal{T}_i$.

Step ① Register Request

$\triangleright$ run by the new user

- 1: **function** RegPrepare(s)
- 2: **run** $\{s_i\} \leftarrow \text{SSS.Share}(s, n, t) \triangleright$ no single entity in domain $\mathcal{D}_u$ can learn s
- 3: **compute** $com \leftarrow g^s$ $\triangleright$ Feldman commitment to user's identity
- 4: **output** $regReq := (\{s_i\}, com) \triangleright$ send $regReq$ to the domain $\mathcal{D}_u$ committee

Step ② Register Response

$\triangleright$ run in a distributed manner by all members C_i of the user's home domain $\mathcal{D}_u$

- 5: **function** RegIssue($\{csk_i\}, regReq$)
- 6: **parse** $regReq$ as (s_i, com)
- 7: **if** $com \in \mathcal{T}_i$ **then output** $\perp$ $\triangleright$ enforce identity uniqueness
- 8: **add** (s_i, com) to $\mathcal{T}_i$
- 9: **compute** $\mu_i \leftarrow csk_i + s_i$
- 10: **generate** random ρ jointly in $\mathcal{D}_u$ $\triangleright$ each C_i in $\mathcal{D}_u$ only obtains share ρ_i
- 11: **obtain** $(\mu\rho)_i$ from μ_i and ρ_i $\triangleright$ run secret sharing multiplication protocol[2]
- 12: **interpolate** $\mu\rho$ from $\{(\mu\rho)_i\}$ $\triangleright \mu\rho = \sum_{i=1}^t (\lambda_i (\mu\rho)_i)$
- 13: **compute** $(\frac{1}{\mu})_i \leftarrow \frac{\rho_i}{\mu\rho}$
- 14: **compute** $usk_i \leftarrow g^{(\frac{1}{\mu})_i}$, $\widetilde{usk}_i \leftarrow \tilde{g}^{(\frac{1}{\mu})_i}$
- 15: **output** $ursk_i := (usk_i, \widetilde{usk}_i)$ $\triangleright$ send $ursk_i$ to the user

Step ③ Secret Key Reconstruction and Verification

$\triangleright$ run by the registered user locally

- 16: **function** USKVerRecon($\{ursk_i\}, cvk^{(\mathcal{D}_u)}$)
- 17: **interpolate** $ursk$ from $\{ursk_i\}$ $\triangleright ursk = (usk, \widetilde{usk}) = (g^{\frac{1}{s+csk}}, \tilde{g}^{\frac{1}{s+csk}})$
- 18: **parse** $ursk$ as $(usk, \widetilde{usk})$
- 19: **if** $e(cvk^{(\mathcal{D}_u)}, g^s, usk) = e(g, \tilde{g})$ and $e(usk, \tilde{g}) = e(g, \widetilde{usk})$ **then**
- 20: **output** $ursk$ $\triangleright$ indicate the reconstructed secret key $ursk$ is valid
- 21: **else output** $\perp$ $\triangleright ursk$ is invalid, so collect other key shares

Step ①: Registration Request. First, a new user initiates the registration process via the function RegPrepare (line 1). The user takes their unique, real-world identity s splits it into n shares using a (t, n) secret sharing scheme (line 2). This ensures that no single member in domain committee $\mathcal{D}_u$ can learn the user's full identity s . Simultaneously, the user computes a Feldman commitment $com = g^s$ to identity s to enable the uniqueness of the new user's identity registration (line 3). Then, the user sends a registration request $regReq$, containing the shares and the commitment, to their home domain committee $\mathcal{D}_u$ (Line 4).

Step ②: Registration Response. The home domain committee then executes the function RegIssue in a distributed manner (line 5). Upon receiving the registration request $regReq$ from the user, all members in $\mathcal{D}_u$ first parse it to extract the corresponding identity shares $\{s_i\}$ and commitment com (line 6). To prevent duplicate registrations, each member C_i checks if com already exists in their local registration table $\mathcal{T}_i$. If it does, the request is rejected and the registration process aborts (line 7). Otherwise, they proceed. Each member C_i records the new user's registration information (s_i, com) to its local registration table $\mathcal{T}_i$, which stores the user's identity share s_i and associated commitment com (line 8). Next, the domain committee issues user registration secret key shares in a distributed manner (line 9-15). Specifically, each committee member C_i first computes an intermediate value $\mu_i = csk_i + s_i$ by combining its secret key share csk_i with the user's identity share s_i

(line 9). To compute the inverse share and blind the committee's secret key, all committee members jointly generate a random value ρ (line 10), where each G_i only gets its own share ρ_i . Then they use a secret sharing multiplication protocol [2] to compute shares of the product $\mu\rho$ (line 11). From these shares, the product can be interpolated (line 12). Each member then computes a share of the inverse, $(\frac{1}{\mu})_i$ (line 13), and uses it to generate a share of the user's secret key, $ursk_i = (usk_i, \widetilde{usk}_i)$ (line 14). These shares are sent to the user (line 15).

Step ③: Secret Key Reconstruction and Verification. Finally, the user runs the function `USKVerRecon` locally (line 16). After collecting at least t key shares, the user interpolates them to reconstruct the full user secret key $ursk = (usk, \widetilde{usk})$ (line 17-18), where it satisfies the following relation:

$$ursk = g^{\frac{1}{s+csk}}, \widetilde{usk} = \widetilde{g}^{\frac{1}{s+csk}}$$

To ensure this key is generated correctly, the user performs two pairing-based checks using the home domain's public verification key $cvk^{(D_u)}$ (line 18-19). These checks are as follows:

$$e(cvk^{(D_u)} \cdot g^s, \widetilde{usk}) = e(g, \widetilde{g})$$

$$e(usk, \widetilde{g}) = e(g, \widetilde{usk})$$

If the checks pass, the reconstructed secret key $ursk$ is valid and the registration is completed (line 20).

4.3 Domain-Hiding Presentation

This phase allows a registered user to anonymously access services across any domain without revealing their home domain or specific identity. To initiate this process, a user first obtains a valid anonymous credential $cred$, from the credential issuers within the DAC framework, as shown in our system workflow (Figure 1, Step ④ Obtain credential). With this credential, the user can generate a domain-hiding presentation to prove their legitimacy to any verifier. Algorithm 3 shows the detailed interactive process, comprising two main functions: `TokenPresent` and `VerPresent`.

Algorithm 3 Domain-Hiding Presentation

Init: each domain D_j publishes its verification key $cvk^{(D_j)}$.
Init: a user registered in domain D_j holds secret key $ursk^{(D_j)}$.
Let the user locally forms a ring $ring = \{cvk^{(D_1)}, \dots, cvk^{(D_k)}\}$ by selecting a set of public domain keys, where one corresponds to the user's home domain.

Step ⑤ Token Presentation
 ▶ run by the user

- 1: **function** `TokenPresent`($ursk^{(D_j)}$, $cred$, $ring$, $cvk^{(D_j)}$, opk)
- 2: **run** $token \leftarrow DAC.Show(cred)$
- 3: **parse** $ursk$ as $(usk, \widetilde{usk})$
- 4: **compute** $T_s \leftarrow e(token, \widetilde{usk})$ ▶ intra-domain tracing tag
- 5: **compute** $c \leftarrow TPKE.Enc(opk, cvk^{(D_j)})$ ▶ encrypt domain index
- 6: **set** relationship $\mathcal{R}_1 := \{cvk^{(D_j)} \in ring\}$ ▶ prove ring membership
- 7: **set** relationship $\mathcal{R}_2 := \{T_s = e(token, \widetilde{usk})\}$ ▶ prove tag is well-constructed
- 8: **set** relationship $\mathcal{R}_3 := \{e(cvk^{(D_j)} \cdot g^s, \widetilde{usk}) = e(g, \widetilde{g})\}$ ▶ prove registration
- 9: **set** relationship $\mathcal{R}_4 := \{c = TPKE.Enc(opk, cvk^{(D_j)})\}$ ▶ prove c is correct
- 10: **generate** proof $\pi_\sigma \leftarrow NIZK.Prove((s, ursk, cvk^{(D_j)}) : \mathcal{R}_1 \wedge \mathcal{R}_2 \wedge \mathcal{R}_3 \wedge \mathcal{R}_4)$
- 11: **output** $(token, \sigma := (T_s, c, \pi_\sigma))$

Step ⑥ Presentation Verification
 ▶ run by the verifier

- 12: **function** `VerPresent`($token$, σ , $ring$, opk)
- 13: **parse** σ as (T_s, c, π_σ)
- 14: **if** `DAC.Verify`($token$) = 1 and `NIZK.Ver`($(T_s, c, ring, opk), \pi_\sigma$) = 1 **then**
- 15: **output** 1 ▶ both the token and signature are valid
- 16: **else output** 0 ▶ verification fails

Step ⑤: Token Presentation. A user initiates the presentation process by invoking function `TokenPresent` (line 1). The core of this function is to generate a one-time unlinkable presentation consisting of a token $token$ and an accountable ring signature σ . This is achieved through the following steps: First, the user generates a one-time show token $token$ from their anonymous credential $cred$ using `DAC.Show` (line 2). Then the user constructs a signature $\sigma = (T_s, c, \pi_\sigma)$ over $token$ (line 4-10). This signature is carefully designed not only to prove the validity of the presentation with hiding the user's home domain in a ring of domains but also to embed information for accountable tracing.

Specially, it comprises two main components. First, an intra-domain tracing tag $T_s = e(token, \widetilde{usk})$ is generated (line 4). It creates a deterministic link between the presentation and the user's secret key, enabling the home domain to identify the user later. Since $token$ is a pseudorandom one-time token, different presentations generate different tags, ensuring unlinkability between multiple service accesses by the same user. Second, a ciphertext $c = TPKE.Enc(opk, cvk^{(D_j)})$ is generated via `TPKE` (line 5). The user's home domain's public key $cvk^{(D_j)}$ is encrypted by the opening committee's public key opk . This ciphertext serves as a trapdoor for cross-domain tracing, allowing the opening committee to identify the user's home domain in a distributed manner.

Furthermore, to prove the well-formed presentation without revealing any secrets (user identity s , secret key $ursk$ and the home domain D_j), the user constructs a NIZK proof, π_σ (Line 10). The proof demonstrates the simultaneous satisfaction of four logical relationships together (lines 6-9):

- $\mathcal{R}_1$: Prove that the user's home domain $cvk^{(D_j)}$ is a member of the public ring of domains. This establishes ring membership.
- $\mathcal{R}_2$: Prove that the tag T_s is correctly constructed from the $token$ and the user's secret key. This proves the tag is well-formed.
- $\mathcal{R}_3$: Prove that the user is truly registered in her home domain, possessing a valid registration secret key corresponding to $cvk^{(D_j)}$, by satisfying the registration verification equation.
- $\mathcal{R}_4$: Prove that the ciphertext c is a valid encryption of user's home domain key under the opening committee's public key.

Finally, the user locally outputs the $token$ along with the signature $\sigma = (T_s, c, \pi_\sigma)$ for presentation (line 11).

Step ⑥: Presentation Verification. Upon receiving the presentation, the verifier executes the function `VerPresent` locally (line 12). The verifier parses the signature σ (line 13) and performs two checks: verifying the one-time show token $token$ with `DAC.Verify` and verifying the NIZK proof π_σ to ensure the signature σ is well-formed and all claims are valid (line 14). If both are valid, the presentation is accepted (line 15) and access to the services is granted to the user. Otherwise, it is rejected (line 16).

4.4 Cross-Domain Tracing

In cases of misuse, ShadowClone provides a hierarchical cross-domain tracing mechanism to enforce accountability. This process, detailed in Algorithm 4, is designed in two stages, minimizing privacy exposure. The first stage identifies the user's home domain, followed by a second stage that opens the user's real-world identity. **Step ⑦: Domain Identification.** This first stage is executed by the opening committee when a suspicious presentation ($token, \sigma$) is reported. First, the opening committee verifies the validity of the

presentation requested to be traced (line 1). If valid, they proceed. Each committee node $\mathcal{P}_i$ uses secret key share osk_i to compute a decryption share ds_i of the ciphertext c from the signature σ , using threshold decryption TPKE.DecShare (line 2-3). To ensure accountability, each node $\mathcal{P}_i$ also generates a corresponding NIZK proof π_{ds_i} attesting to the correct computation of its decryption share (line 4). After collecting a threshold of t_o valid decryption shares and their valid proofs (line 5), the committee combines the shares using TPKE.Combine. This recovers the plaintext, which is the public verification key of the malicious user's home domain, $cvk^{(\mathcal{D}_c)}$ (line 6). Next, the committee constructs a package proof π_{open} to ensure the verifiability of this domain opening process. Then, the committee sends a TRACE message containing the opened verification key $cvk^{(\mathcal{D}_c)}$ and this proof π_{open} to the identified domain committee $\mathcal{D}_c$ (line 8), initiating the second stage.

Algorithm 4 Hierarchical Cross-Domain Tracing

Init: each domain $\mathcal{D}_j$ publishes its verification key $cvk^{(\mathcal{D}_j)}$.
Init: a user registered in her home domain $\mathcal{D}_i$ with her identity s .
Init: tracing is triggered by suspicious credential usage with evidence (token, σ) .
Init: opening authorities hold secret keys $\{osk_j\}$ and public key opk for TPKE.

Step ⑦ Domain Identification ▷ run by opening committee
 ▶ As a node $\mathcal{P}_i$ in opening committee:
 1: if $\text{VerPresent}(\text{token}, \sigma, \text{ring}, opk) = 0$ then output $\perp$
 2: parse σ as (T_s, c, σ)
 3: compute $ds_i \leftarrow \text{TPKE.DecShare}(osk_i, c)$ ▷ threshold decryption
 4: generate proof $\pi_{ds_i} \leftarrow \text{NIZK.Prove}(osk_i : ds_i = \text{TPKE.DecShare}(osk_i, c))$
 5: obtain t_o valid decrypted shares $\{ds_j\}$ from opening committee
 6: compute $cvk^{(\mathcal{D}_c)} \leftarrow \text{TPKE.Combine}(\{ds_j\})$ ▷ identify user's home domain
 7: set $\pi_{open} := (\{ds_j\}, c, \{\pi_{ds_j}\})$ ▷ make the opening verifiable
 8: send $(\text{TRACE}, cvk^{(\mathcal{D}_c)}, \pi_{open})$ to $\mathcal{D}_c$

Step ⑧ Intra-Domain Tracing ▷ run by domain committee
 ▶ As a node $\mathcal{C}_i$ in the identified domain committee $\mathcal{D}_c$:
 9: upon receiving valid TRACE message from opening committee do
 10: compute $T_{tk} \leftarrow e(\text{token}, \bar{g})$
 11: for each entry $\langle k \rangle$ in the registration table of $\mathcal{C}_i$ do
 12: compute $[k] \leftarrow s_i^{(k)} + csk_i, T_i^{(k)} \leftarrow T_s^{[k]}$
 13: obtain t valid shares $\{T_j^{(k)}\}$ from $\mathcal{D}_c$
 14: interpolate $T^{(k)}$ from $\{T_j^{(k)}\}$
 15: if $T^{(k)} = T_{tk}$ then ▷ find the user, end search loop
 16: run $s \leftarrow \text{SSS.Reconstruct}(\{s_i^{(k)}\}_{i \in [t]})$ ▷ opening the user's identity
 17: generate $\pi_{T_i} \leftarrow \text{NIZK.Prove}(\{s_i^{(k)}, csk_i, [k] : T_i^{(k)} = T_s^{[k]}\})$
 18: output s and $\pi_{trace} := (\{T_i^{(k)}\}, T_{tk}, \{\pi_{T_i}\})$

Step ⑧: Intra-Domain Tracing. This second stage is performed by the identified domain committee $\mathcal{D}_c$. Upon receiving a valid TRACE message (line 9) from the opening committee, the committee $\mathcal{D}_c$ members begin the process to identify the malicious user in $\mathcal{D}_c$ responsible for the misuse. First, they compute a target tag $T_{tk} = e(\text{token}, \bar{g})$ using the public token token provided in the trace request (line 10). This T_{tk} serves as a flag for identifying the malicious user in the search. Next, the committee iterates through each registered user entry $\langle k \rangle$ in their local registration tables (line 11). For each user, it performs a collaborative computation to check for a match (line 12-15). Each member $\mathcal{C}_i$ first computes a partial tag $T_i^{(k)} = T_s^{s_i^{(k)} + csk_i}$ using its secret key share csk_i and the user's identity share $s_i^{(k)}$ (line 12). These partial tags are then interpolated to obtain the complete candidate tag $T^{(k)}$ (line 13-14). This candidate tag $T^{(k)}$ is compared with the target tag T_{tk} . If they match, the correct user has been found and the search loop ends

(line 15). Once identified, the committee reconstructs her real-world identity s by interpolating the corresponding identity shares (line 16). During the same time, each member generates a NIZK proof for its contribution. These proofs are then packaged into a tracing proof π_{trace} to ensure public verifiability of this tracing process (line 17). Finally, the committee outputs the malicious user's identity s along with the tracing proof, completing the tracing process (lines 18).

5 Implementation and Experiments

5.1 Performance Evaluation

Experimental Settings. We implement a prototype of ShadowClone to evaluate the performance. The experiments are conducted on the machine equipped with an Apple M2 SoC, which operates with macOS 15.5, featuring 8-core CPU and 16GB UM.

Implementation details. We implement the algorithms described in Section 4 in Golang using the PBC library¹. The bilinear pairing is the D-type pairing defined in PBC and set to an approximate ECC-200-bit security level. For the DAC component, we use scheme [33] to instantiate, where the user credential is fixed to 10 attributes. The TPKE is instantiated with an ElGamal-based construction [10]. We conduct an evaluation on each part runtime cost of our system and compare it with a baseline DID system FADID-TT [27] (WWW'25) that provides anonymity within a single domain.

Performance of ShadowClone. We first characterize the performance of ShadowClone's core operations. Figure 2a shows that the runtime of the setup, registration and tracing phases, which involve distributed cryptographic protocols like secret sharing, increase with committee size. The user-centric presentation phase, executed locally in user-side, remains unaffected by committee size. Figure 2b explores different sharding configurations with a fixed total of 100 committee members. As shown, a higher degree of sharding (i.e., more committees of a smaller size) reduces the time for setup and registration. The corresponding increase in presentation cost is an expected trade-off, as the user generates a more complex proof to hide her home domain within a larger anonymity set (e.g., 10 domains instead of 4), thus achieving stronger privacy.

Overall performance comparison. Figure 2c presents the performance comparison between ShadowClone, configured with 3 domain committees of (10, 4), and the monolithic FADID-TT, configured as (30, 12). Figure 2c demonstrates the efficiency of ShadowClone. Specifically, it reduces the setup time by 33.2% (from 268ms to 179ms) and the registration time by 64.6% (from 271ms to 96ms). The tracing is also faster, dropping from 101ms to 55ms. Besides, the ShadowClone can offer strong cross-domain anonymity with an acceptable increase in presentation cost.

Scalability in setup. Figure 3a measures the setup time as the total number of committee members grows. FADID-TT's monolithic setup time scales poorly, increasing from 243ms to 534ms. In contrast, ShadowClone's setup time shows much flatter growth (from 181ms to 264ms). This validates that its sharding architecture, which enables parallel key generation in smaller committees, effectively mitigates setup overhead in large-scale systems.

Scalability in registration. Figure 3b assesses registration scalability by measuring the registration time as the number of users increases. FADID-TT's performance degrades sharply, taking nearly

¹<https://github.com/Nik-U/pbc>

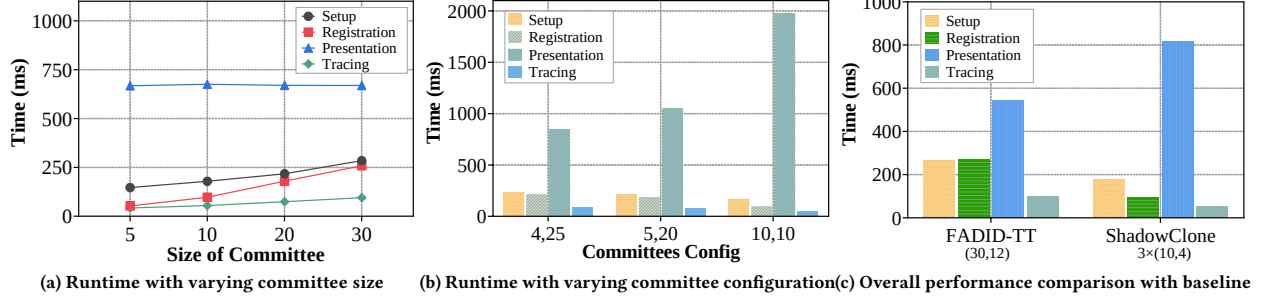


Figure 2: Performance of ShadowClone

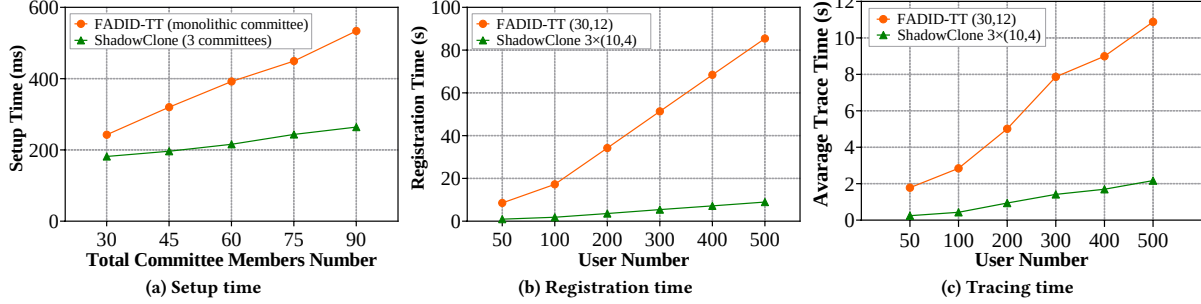


Figure 3: Scalability of ShadowClone compared with the baseline FADID-TT

85.4 sec to register 500 users due to its serial processing. ShadowClone exhibits scalability, completing the same task in 8.9 sec, nearly $10\times$ faster. This is because user registration requests are distributed in small committees and processed in parallel across multiple domain committees, reducing the computational complexity from a linear $O(m)$ in the baseline to more scalable $O(m/k)$, where m is the number of users and k is the number of domains.

Tracing efficiency. Figure 3c compares the average tracing time as the user registry expands. FADID-TT's average tracing time, which requires a full scan of the user list, grows linearly to approximately 11 sec for 500 users. ShadowClone's time increases at a much lower rate, taking only 2.2 sec for the same number of users. This result confirms the efficiency of our hierarchical tracing mechanism, which first identifies the malicious user's home domain and then performs a fast, localized search, reducing the complexity from a global $O(m)$ search to a far more efficient local $O(m/k)$ search.

5.2 Deployment on Blockchain

We deploy ShadowClone and FADID-TT on two representative blockchain platforms, including Hyperledger Fabric (permissioned setting) and Ethereum (permissionless setting), to demonstrate the practical feasibility with existing distributed infrastructure. The global committee members is fixed as 30. FADID-TT adopts the configuration of $(n, t) = (30, 12)$, while ShadowClone has 3 domain committees (each with $(10, 4)$). All registration and presentation procedures are executed off-chain, while the setup (recording domain public keys) and tracing (publishing malicious users' real identities and tracing proofs) phases involve on-chain operations.

As Table 2 shows, ShadowClone costs around 2.08-2.31 sec per on-chain operation on Fabric and 71.8k-86.0k gas on Ethereum, reducing the on-chain overhead compared to FADID-TT. This is owing to that ShadowClone has fewer participants in tracing, thus reducing tracing proof size and on-chain cost. Besides, the sharding architecture allows parallel setup across domains, further reducing

end-to-end latency. The results demonstrates ShadowClone has high efficiency in the practical deployment of multi-domain Web3.

Table 2: Comparison of time/gas cost on the Fabric/Ethereum

Operation	FADID-TT [27]		ShadowClone	
	Fabric (time)	Ethereum (gas)	Fabric (time)	Ethereum (gas)
Setup	5.83s	92470	2.31s	86038
Registration	271ms (off-chain)		96ms (off-chain)	
Presentation	545ms (off-chain)		818ms (off-chain)	
Tracing	5.51s	92652	2.08s	71852

6 Conclusion

We present ShadowClone, a scalable decentralized identity system that provides strong cross-domain anonymity and accountable traceability in the multi-domain Web3 ecosystem. By integrating a sharding architecture with a novel domain-hiding presentation protocol and a hierarchical tracing mechanism, ShadowClone safeguards user privacy against cross-domain profiling while ensuring efficient, publicly verifiable accountability for malicious behavior. Our implementation and evaluation demonstrate that ShadowClone improves scalability and tracing efficiency compared to the baseline FADID-TT system, making it a practical foundation for next-generation identity management in the Web3 multiverse.

Acknowledgments

This paper is supported by the National Key R&D Program of China (2025ZD0808500), the National Natural Science Foundation of China (T2425023, U21B2021, U22B2008, U21A20467, U2241213, 62202027, 62172025, 62472015), Beijing Natural Science Foundation (L251003), Zhejiang Provincial Natural Science Foundation of China under Grant No. LMS25F020014, Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University, the Fundamental Research Funds for the Central Universities, Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing.

References

- [1] Lennart Ante and Ingo Fiedler. 2025. The new digital economy: How decentralized finance (DeFi) and non-fungible tokens (NFTs) are transforming value creation, ownership models, and economic systems. , 100094 pages.
- [2] Gilad Asharov and Yehuda Lindell. 2017. A Full Proof of the BGW Protocol for Perfectly Secure Multiparty Computation. *J. Cryptol.* 30, 1 (2017), 58–151.
- [3] Oscar Avellaneda, Alan Bachmann, Abbie Barbir, Joni Brennan, Pamela Dingle, Kim Hamilton Duffy, Eve Maler, Drummond Reed, and Manu Sporny. 2019. Decentralized Identity: Where Did It Come From and Where Is It Going? *IEEE Commun. Stand. Mag.* 3, 4 (2019), 10–13.
- [4] Manuel Blum, Paul Feldman, and Silvio Micali. 2019. Non-interactive zero-knowledge and its applications. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, Oded Goldreich (Ed.). ACM, 329–349.
- [5] Ran Canetti and Shafi Goldwasser. 1999. An efficient threshold public key cryptosystem secure against adaptive chosen ciphertext attack. In *Proceedings of the 17th International Conference on Theory and Application of Cryptographic Techniques* (Prague, Czech Republic) (EUROCRYPT'99). Springer-Verlag, Berlin, Heidelberg, 90–106.
- [6] Rutchathon Chairattana-Apirom, Franklin Harding, Anna Lysyanskaya, and Stefano Tessaro. 2025. Server-Aided Anonymous Credentials. In *Advances in Cryptology – CRYPTO 2025*. Springer Nature Switzerland, Cham, 291–324.
- [7] Melissa Chase, Chaya Ganesh, and Payman Mohassel. 2016. Efficient Zero-Knowledge Proof of Algebraic and Non-Algebraic Statements with Applications to Privacy Preserving Credentials. In *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 9816)*. Springer, 499–530.
- [8] Hanfang Chen, Niankun Wei, Leyao Wang, Wael Fawzy Mohamed Mobarak, Marwan Ali Albahar, and Zaffar Ahmed Shaikh. 2024. The Role of Blockchain in Finance Beyond Cryptocurrency: Trust, Data Management, and Automation. *IEEE Access* 12 (2024), 64861–64885.
- [9] Cheng Chi, Daozhuang Lin, Ramesh Ramadoss, Yu Yuan, Zihang Yin, Yuwen Zhang, Tongtong Cheng, Wenqu Chen, Lingyuan Yin, Bin Yang, et al. 2023. White Paper-Global Web3 Technology Industry Ecological Development Report. *Global Web3 Technology Industry Ecological Development Report* (2023), 1–22.
- [10] Yvo G. Desmedt and Yair Frankel. 1989. Threshold cryptosystems. In *Proceedings on Advances in Cryptology* (Santa Barbara, California, USA) (CRYPTO '89). Springer-Verlag, Berlin, Heidelberg, 307–315.
- [11] Bidita Sarkar Diba, Jayanto Dutta Plabon, Nishat Tasnim, Mehjabin Hossain, Durjoy Mistry, Sourav Sarker, M. F. Mridha, Yuichi Okuyama, and Jungpil Shin. 2025. From Centralization to Decentralization: Blockchain's Role in Transforming Social Media Platforms. *IEEE Access* 13 (2025), 80478–80507.
- [12] DIF. 2022. Together we're building a new identity ecosystem. <https://identity.foundation>. Accessed Jan. 23, 2024.
- [13] DIF. 2025. ION - an open, public, permissionless decentralized identifier network. <https://identity.foundation/ion/>. Accessed Oct. 08, 2025.
- [14] Jack Doerner, Yashvanth Kondi, Eysa Lee, Abhi Shelat, and LaKyah Tyner. 2023. Threshold BBS+ Signatures for Distributed Anonymous Credential Issuance. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 773–789.
- [15] Endless Domains. 2025. The Expanding Ecosystem of Web3 TLDs: From .eth to .dao, What's Next? <https://endlessdomains.io/blog/posts/web3-tlds-digital-identity-ownership>
- [16] Paul Dunphy and Fabien A. P. Petitcolas. 2018. A First Look at Identity Management Schemes on the Blockchain. *IEEE Secur. Priv.* 16, 4 (2018), 20–29.
- [17] Andrea Flamini, Eysa Lee, and Anna Lysyanskaya. 2025. Multi-Holder Anonymous Credentials from BBS Signatures. In *Advances in Cryptology – CRYPTO 2025*. Springer Nature Switzerland, Cham, 325–357.
- [18] Steven D. Galbraith, Kenneth G. Paterson, and Nigel P. Smart. 2008. Pairings for cryptographers. *Discret. Appl. Math.* 156, 16 (2008), 3113–3121.
- [19] Christina Garman, Matthew Green, and Ian Miers. 2014. Decentralized Anonymous Credentials. In *21st Annual Network and Distributed System Security Symposium, NDSS 2014, San Diego, California, USA, February 23-26, 2014*. The Internet Society.
- [20] Oded Goldreich and Yair Oren. 1994. Definitions and Properties of Zero-Knowledge Proof Systems. *J. Cryptol.* 7, 1 (1994), 1–32.
- [21] Hasini Gunasinghe, Ashish Kundu, Elisa Bertino, Hugo Krawczyk, Suresh Chari, Kapil Singh, and Dong Su. 2019. PrivdEx: Privacy Preserving and Secure Exchange of Digital Identity Assets. In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, Ling Liu, Ryan W. White, Amin Mantrach, Fabrizio Silvestri, Julian J. McAuley, Ricardo Baeza-Yates, and Leila Zia (Eds.). ACM, 594–604.
- [22] Chloé Héban and David Pointcheval. 2022. Traceable Constant-Size Multi-authority Credentials. In *Security and Cryptography for Networks - 13th International Conference, SCN 2022, Amalfi, Italy, September 12-14, 2022, Proceedings (Lecture Notes in Computer Science, Vol. 13409)*. Springer, 411–434.
- [23] Hyperledger. 2019. Hyperledger Aries. <https://www.hyperledger.org/projects/aries>. Accessed: 2025-10-08. Hyperledger Aries is an open-source framework for building interoperable, decentralized identity solutions..
- [24] Daiki Ito, Yuta Takata, Hiroshi Kumagai, and Masaki Kamizono. 2024. Investigations of Top-Level Domain Name Collisions in Blockchain Naming Services. In *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, Tat-Seng Chua, Chong-Wah Ngo, Ravi Kumar, Hady W. Lauw, and Roy Ka-Wei Lee (Eds.). ACM, 2926–2935.
- [25] Polygon Labs. 2025. Polygon ID. <https://www.privado.id/>. Accessed Oct. 08, 2025.
- [26] Yizhong Liu, Boyu Zhao, Zedan Zhao, Jianwei Liu, Xun Lin, Qianhong Wu, and Willy Susilo. 2024. SS-DID: A Secure and Scalable Web3 Decentralized Identity Utilizing Multilayer Sharding Blockchain. *IEEE Internet Things J.* 11, 15 (2024), 25694–25705.
- [27] Yizhong Liu, Zedan Zhao, Boyu Zhao, Feiang Ran, Xun Lin, Dawei Li, and Zhenyu Guan. 2025. Fully Anonymous Decentralized Identity Supporting Threshold Traceability with Practical Blockchain. In *Proceedings of the ACM on Web Conference 2025 (Sydney NSW, Australia) (WWW '25)*. Association for Computing Machinery, New York, NY, USA, 3628–3638.
- [28] Deepak Maram, Harjasleen Malvai, Fan Zhang, Nerla Jean-Louis, Alexander Frolov, Tyler Kell, Tyrone Lobban, Christine Moy, Ari Juels, and Andrew Miller. 2021. Candid: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability. In *2021 IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, May 24-27, 2021*. IEEE, 1348–1366.
- [29] Victor S. Miller. 2004. The Weil Pairing, and Its Efficient Calculation. *J. Cryptol.* 17, 4 (2004), 235–261.
- [30] Omid Mir, Balhazar Bauer, Scott Griffy, Anna Lysyanskaya, and Daniel Slamanig. 2023. Aggregate Signatures with Versatile Randomization and Issuer-Hiding Multi-Authority Anonymous Credentials. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*. ACM, 30–44.
- [31] Nitin Naik and Paul Jenkins. 2021. Sovrin network for decentralized digital identity: Analysing a self-sovereign identity system based on distributed ledger technology. In *2021 IEEE International Symposium on Systems Engineering (ISSE)*. IEEE, 1–7.
- [32] Hye-young Paik, Yue Liu, Qinghua Lu, and Salil S Kanhere. 2023. Decentralized Identity Management and Blockchains: Design Patterns and Architectures. In *Blockchains: A Handbook on Fundamentals, Platforms and Applications*. Springer, 465–491.
- [33] David Pointcheval and Olivier Sanders. 2016. Short Randomizable Signatures. In *Topics in Cryptology - CT-RSA 2016 - The Cryptographers' Track at the RSA Conference 2016, San Francisco, CA, USA, February 29 - March 4, 2016, Proceedings (Lecture Notes in Computer Science, Vol. 9610)*. Springer, 111–126.
- [34] Juliane Proelss, Stéphane Sévigny, and Denis Schweizer. 2023. GameFi: The perfect symbiosis of blockchain, tokens, DeFi, and NFTs? *International Review of Financial Analysis* 90 (2023), 102916.
- [35] Partha Pratim Ray. 2023. Web3: A comprehensive review on background, technologies, applications, zero-trust architectures, challenges and future directions. *Internet of Things and Cyber-Physical Systems* (2023).
- [36] Michael Rosenberg, Jacob D. White, Christina Garman, and Ian Miers. 2023. zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 790–808.
- [37] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613.
- [38] Victor Shoup and Rosario Gennaro. 1998. Securing threshold cryptosystems against chosen ciphertext attack. In *Advances in Cryptology – EUROCRYPT'98*, Kaisa Nyberg (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–16.
- [39] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. 2019. Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers. In *26th Annual Network and Distributed System Security Symposium, NDSS2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society.
- [40] Xinxin Xing, Yizhong Liu, Boyang Liao, Jianwei Liu, Bin Hu, Xun Lin, Yuan Lu, and Tianwei Zhang. 2025. GoSSamer: Lightweight and Linear-Communication Asynchronous (Dynamic Proactive) Secret Sharing and the Applications. *IACR Cryptol. ePrint Arch.* (2025), 1516.
- [41] Ruoting Xiong, Wei Ren, Xiaohan Hao, Jie He, and Kim-Kwang Raymond Choo. 2023. BDIM: A Blockchain-Based Decentralized Identity Management Scheme for Large Scale Internet of Things. *IEEE Internet Things J.* 10, 24 (2023), 22581–22590.

A Detailed Cryptographic Preliminaries

A.1 Bilinear Pairing

Our cryptographic constructions are built upon bilinear pairings. Let $\mathbb{G}$, $\bar{\mathbb{G}}$, and $\mathbb{G}_T$ be three finite groups of prime order p . Let g be

a generator of $\mathbb{G}$ and $\tilde{g}$ be a generator of $\tilde{\mathbb{G}}$. A bilinear pairing is a map $e: \mathbb{G} \times \tilde{\mathbb{G}} \rightarrow \mathbb{G}_T$ that satisfies the following properties [29].

- **Bilinearity:** For all $u \in \mathbb{G}$, $v \in \tilde{\mathbb{G}}$ and for all $a, b \in \mathbb{Z}_q^*$, we have $e(u^a, v^b) = e(u, v)^{ab}$;
- **Non-degeneracy:** $e(g, \tilde{g}) \neq 1_{\mathbb{G}_T}$, where $1_{\mathbb{G}_T}$ is the identity element in $\mathbb{G}_T$;
- **Computability:** There exists an efficient algorithm to compute $e(u, v)$ for all $u \in \mathbb{G}$, $v \in \tilde{\mathbb{G}}$.

Our scheme utilizes a Type-3 pairing, where no efficient isomorphism between $\mathbb{G}$ and $\tilde{\mathbb{G}}$ is known [18]. The global bilinear pairing parameters are denoted as $bp = (p, \mathbb{G}, \tilde{\mathbb{G}}, \mathbb{G}_T, e, g, \tilde{g})$.

A.2 Secret Sharing

A (n, t) threshold secret sharing scheme allows a dealer to split a secret s into n shares, where any t or more shares can reconstruct s , but any $t - 1$ or fewer shares reveal no information about it [40]. In ShadowClone, we instantiate this with Shamir's Secret Sharing scheme (SSS) [37], which is the most classic and widely used secret sharing scheme. It consists of the following two algorithms:

- **SSS.Share(s, n, t)** $\rightarrow \{s_i\}_{i=1}^n$: This algorithm takes a secret s , a threshold t , and the number of participants n as input, and outputs n shares $\{s_i\}$. Specifically, the dealer selects $t - 1$ random coefficients $a_1, a_2, \dots, a_{t-1}$ on a finite field $\mathbb{F}_p$ and constructs the polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i$ taking the secret s as the constant term. For each participant i , a corresponding share $s_i = f(i)$ is computed and distributed securely to participant i .
- **SSS.Reconstruct($\{s_i\}_{i \in I}$)** $\rightarrow s$: Given any set of at least t shares, where I is an index set with $|I| \geq t$, this algorithm outputs the original secret s using Lagrange interpolation. For each $i \in I$, the corresponding Lagrange coefficient λ_i is computed as $\lambda_i = \frac{\prod_{j \in I, j \neq i} j}{\prod_{j \in I, j \neq i} (j - i)}$. Then, by interpolation, the secret s can be reconstructed as $s = \sum_{i \in I} \lambda_i s_i$.

In ShadowClone, we use (n, t) SSS to distribute users' identity information among domain committee members, ensuring that no single member can learn the complete identity.

A.3 NIZK Proofs

A zero-knowledge proof (ZKP) allows a prover to convince a verifier that a public statement x is true, without revealing any additional knowledge about the secret witness w that makes x valid under a public known relation $\mathcal{R}$. Interactive ZKPs require multiple rounds of interaction between the prover and the verifier. A Non-Interactive Zero-Knowledge (NIZK) proof removes this need for interaction. In NIZK proof system, the prover generates a single proof string π that can be verified by verifier later, without further communication [4].

A ZKP system, including Non-Interactive Zero-Knowledge (NIZK) proof system, must satisfy three fundamental properties [20]:

- **Completeness.** If the statement is true, an honest prover possessing a valid witness can always successfully convince the verifier.
- **Soundness.** If the statement is false, no dishonest prover can convince the verifier of the statement's truth, except with a negligible probability.
- **Zero-knowledge.** The interaction between the prover and the verifier only reveals the fact that the statement is true, without disclosing any additional knowledge about the secret witness.

In ShadowClone, we abstract the core functionalities of the NIZK into two algorithms [6]: NIZK.Prove and NIZK.Verify. We instantiate with Schnorr-style proofs of knowledge for discrete logarithm relations [7], which are efficient and well established.

- **NIZK.Prove($w : \mathcal{R}$)** $\rightarrow \pi$: This is proving algorithm, executed by the prover. We omit the public inputs for simplicity here. The algorithm outputs a proof π showing that the secret witness w , together with the public parameters, satisfies the relation $\mathcal{R}$.
- **NIZK.Verify(π)** $\rightarrow \{0, 1\}$: This is the verification algorithm, executed by the verifier. It outputs 1 (true) if the proof π is valid, and 0 (false) otherwise.

A.4 Threshold Public Key Encryption

Threshold Public Key Encryption (TPKE) is a critical component for our accountable tracing mechanism. A TPKE scheme [5, 10, 38] is a threshold public key cryptosystem where the secret key is distributed among multiple parties, and a threshold of these parties must cooperate to decrypt a ciphertext. A TPKE scheme consists of the following algorithms:

- **TPKE.KeyGen($1^\lambda, t, n$)** $\rightarrow (pk, \{sk_i\}_{i=1}^n)$: A distributed key generation protocol run by n parties to produce a single public encryption key pk and n secret decryption key shares $\{sk_i\}$, with a decryption threshold t .
- **TPKE.Enc(pk, m)** $\rightarrow c$: An algorithm to encrypt a plaintext message m under the public key pk , producing a ciphertext c .
- **TPKE.DecShare(sk_i, c)** $\rightarrow ds_i$: A party i uses its secret key share sk_i to generate a partial decryption share ds_i for ciphertext c .
- **TPKE.Combine($\{ds_i\}_{i \in I}$)** $\rightarrow m$: Given a set of at least t valid decryption shares, this algorithm reconstructs and outputs the original plaintext message m .

A.5 Decentralized Anonymous Credentials

A Decentralized Anonymous Credential (DAC) system [14, 19, 30] allows users to obtain credentials from distributed issuers and later prove ownership of these credentials anonymously without revealing unnecessary personal data. An advantage of the DACs paradigm is the elimination of central intermediaries for verification, mitigating the risk of single points of failure. Furthermore, DACs are privacy-preserving, supporting core functionalities like selective disclosure of user attributes and unlinkable authentication. The core properties we require from DACs are unlinkability, which means that multiple presentations from the same credential cannot be linked. We abstract a DAC scheme using the following algorithms:

- **DAC.Issue($req, \{isk_i\}$)** $\rightarrow cred$: On a user's request req , a set of distributed issuers uses their secret keys $\{isk_i\}$ to collaboratively issue a credential $cred$ to the user.
- **DAC.Show($cred$)** $\rightarrow token$: The user runs this algorithm on their credential $cred$ locally to generate a rerandomized one-time "show token" $token$. The $token$ serves as a proof that the user owns a valid credential approved by the issuers without revealing the actual identity information. The non-deterministic nature of this algorithm ensures that different tokens generated from the same credential are unlinkable.
- **DAC.Verify($token, ipk$)** $\rightarrow \{0, 1\}$: A verifier uses the issuers' public key ipk to check the validity of a $token$. It returns 1 if valid, and 0 otherwise.

B Security Analysis

Here, following the design details in Section 4, we provide a comprehensive security analysis demonstrating that ShadowClone achieves the defined security properties and resists potential attacks.

B.1 Security Definition

We first give the definition of security properties in ShadowClone as follows:

Definition 1 (Strong Cross-Domain Anonymity). We say that a DID system achieves *strong cross-domain anonymity*, if it satisfies the following three properties:

- *Identity anonymity*. A presentation produced by an honest registered user and the correct registration process should not reveal the user’s real identity to any adversary $\mathcal{A}$, even when the adversary controls some committee members (below the opening/tracing thresholds).
- *Home-domain indistinguishability (domain-hiding)*. A presentation should not reveal which domain the user belongs to. Among the set of domains represented in the presentation’s ring, the user’s home domain must be computationally indistinguishable.
- *Unlinkability across presentations*. Multiple presentations produced by the same user (possibly in different verifier domains and at different times) should be computationally unlinkable. This means that an adversary $\mathcal{A}$ should not be able to tell whether two given presentations originate from the same user, beyond negligible advantage.

This definition introduces a more stringent privacy guarantee than the “full anonymity” proposed in prior works like [27]. While [27] ensures unlinkability and anonymity within a single, monolithic committee architecture, it does not address the privacy leakage inherent in a multi-domain context. In such a system, any interaction would reveal the user’s affiliation with the single, unified system (e.g., “this user is from the DeGame”), allowing for domain-level profiling. Our definition of *Strong Cross-Domain Anonymity* closes this critical gap by ensuring a user’s home domain affiliation is cryptographically hidden within an anonymity set of other domains, preventing such profiling and achieving robust state of anonymity in the Web3 multiverse.

Definition 2 (Accountable Traceability). The DID system is said to provide *accountable traceability* if it satisfies the following completeness and soundness properties:

- *Domain-identification completeness*. If a presentation is honestly produced by a registered user from domain D_j and a valid trace request is approved, then the opening committee can recover $cvk^{(D_j)}$ and produce a publicly verifiable opening proof.
- *Intra-domain tracing completeness*. Given the opened domain, the responsible domain committee can recover the user’s identity s .
- *Non-frameability (soundness)*. An adversary $\mathcal{A}$ cannot cause a valid trace that wrongly attributes to an honest user.

This hierarchical and public verifiable approach is more efficient and privacy-preserving than a flat tracing mechanism that would require scanning a global user registry, while verifying the correctness of the whole tracing process and making it accountable.

Definition 3 (Unforgeability). The DID system is said to be *unforgeable* if no probabilistic polynomial-time adversary $\mathcal{A}$, defined in our threat model, can succeed in the following tasks:

- *Registration Unforgeability*. The adversary $\mathcal{A}$ cannot forge a valid user registration secret key ur_{sk} of the honest user.
- *Presentation Unforgeability*. After observing any number of legitimately generated presentations, the adversary $\mathcal{A}$ cannot create a new valid presentation $(stoken', \sigma')$ that is accepted by the verification algorithm, without possessing the legitimate registration secret key and its corresponding credential.

B.2 Security Proof

We now present the core ideas of the security proof, demonstrating that ShadowClone satisfies the security properties defined in B.1.

Throughout the proofs, we rely on the standard hardness of discrete logarithms and bilinear pairings, the security of underlying DACs, the IND-CPA and correctness of TPKE, the secrecy and correctness of Shamir secret sharing, the zero-knowledge, completeness and soundness of NIZKs, and the threat model defined in Section 3.2.

Theorem 1 (Strong Cross-Domain Anonymity). *Assuming DACs, TPKE, and NIZKs are secure and the PPT adversary $\mathcal{A}$ control fewer than the corruption thresholds in both domain and opening committees, ShadowClone achieves strong cross-domain anonymity as defined in Definition 1.*

PROOF. *Identity anonymity.* In registration phase, user’s real identity s is secret-shared among domain committee and only the commitment is published. With fewer than t shares and the hiding property of the commitment, the adversary cannot recover s . In each presentation, the user only discloses a randomized DAC token and an accountable ring signature over a set of domain public keys. The signature is composed of a tag derived from the one-time token, a TPKE ciphertext, and a NIZK proof of well-formedness. If an adversary could recover user’s real identity from the presentation, it would break the anonymity of DACs, the zero-knowledge property of NIZKs, or the semantic security of TPKE.

Domain-hiding. The ciphertext of the user’s home domain key is indistinguishable from encryptions of any other domain under IND-CPA security of TPKE, and the accompanying NIZK guarantees well-formedness without leaking information. Thus, the user’s home domain remains hidden within the anonymity set.

Unlinkability across presentations. Each presentation employs fresh randomness from DACs and produces randomized tokens, tags, ciphertext and NIZKs. Linking two presentations would imply breaking the unlinkability of DACs, randomness of TPKE, or the zero-knowledge of the proofs. Hence, cross-presentation unlinkability holds. $\square$

Theorem 2 (Hierarchical Accountable Tracing). *Assuming TPKE, and NIZKs are secure and the PPT adversary $\mathcal{A}$ control fewer than the corruption thresholds in both domain and opening committees, ShadowClone satisfies accountable traceability, including domain-identification completeness, intra-domain tracing completeness, and non-frameability, as defined in Definition 2.*

PROOF. *Domain identification completeness.* Each presentation embeds a TPKE encryption of the user’s home domain key. Once a valid trace request is approved, the opening committee combines at

least t_o correct decryption shares (verified with NIZKs) to reveal the domain key. This property is ensured by the correctness of TPKE and the completeness of NIZKs.

Intra-domain tracing completeness. Given the opened domain, the responsible committee uses the presentation's intra-domain tag together with stored registration record to recover the user identity. Since shares are consistent and NIZKs ensure correctness, the tracing process deterministically identifies the user.

Non-frameability. Forging a trace that falsely implicates a honest user would require either producing a malformed ciphertext that decrypts to the wrong domain (breaking TPKE or NIZK soundness) or generating a valid tag for another user without their secret key (breaking DAC or signature unforgeability). Neither is possible under our assumptions, ensuring honest users cannot be framed. $\square$

Theorem 3 (Unforgeability). *ShadowClone satisfies unforgeability as defined in Definition 3.*

PROOF. Registration unforgeability. A valid registration secret key ur_{sk} is interpolated from at least t valid shares and must satisfy the pairing checks against the domain committee public key. Therefore, the adversary controlling members fewer than the threshold t , must forge shares to create a new ur_{sk} . This forgery contradicts the discrete logarithm hardness assumptions in the underlying group.

Presentation unforgeability. A valid presentation requires a DAC token, a correct ring signature, and a consistent NIZK. Forging such a tuple without a legitimate registration secret key would directly break the unforgeability of DACs or the soundness of the proof system. Thus, presentations are unforgeable. $\square$

C Related Work

Our work builds upon and extends several key areas in decentralized identity (DID), focusing on privacy, accountability, and scalability. Existing approaches can be broadly categorized as follows.

Decentralized identity and anonymous credentials. Foundational DID systems focused on establishing user-centric control over identifiers. Early systems like Sovrin [31] and ION [13] leveraged blockchain to decentralize identifier registration, primarily addressing availability and verifiability rather than strong user privacy. Subsequent works [23, 25, 27] integrated anonymous credentials to enhance privacy. Coconut [39] by Sonnino et al introduced threshold issuance for selective disclosure credentials, enhancing decentralization distributing trust among multiple issuers, but it does not address accountability for malicious users. CanDID [28], proposed by Maram et al., provides a decentralized identity framework that is compatible with legacy systems and achieves sybil-resistance, but its usage of pairwise DIDs offers weaker privacy guarantees and fails to prevent link analysis across interactions.

Anonymity with accountable traceability. TMAC [22] proposed by Héban and Pointcheval, offers traceable multi-authority credentials, but relies on a centralized authority for tracing, which conflicts with the decentralized ethos of Web3. More recently, FADID-TT [27] by Liu et al. introduced a fully anonymous DID system with threshold verifiable traceability. It achieves this within a single domain using a distributed committee. However, its monolithic architecture and the searching of the entire registry in tracing process present a significant scalability bottleneck. Besides, it does not provide cross-domain anonymity, as a user's affiliation with the system's single domain is implicitly revealed during interactions, enabling potential profiling.

Scalability in DID systems. The performance limitations of monolithic committee mode used by FADID-TT [27] and other earlier DID systems have motivated research into more scalable schemes. SS-DID [26] utilizes a multilayer sharding blockchain to construct a scalable DID framework. However, its primary focus is performance, and it does not provide the sophisticated privacy guarantees, such as anonymity and accountability.